

PENETRATION TEST REPORT

OWASP Juice Shop — Customer-Facing Web Portal

Black-Box Web Application Security Assessment

Submitted by: Vivek Makwana

Internship: Ethical Hacking

Prepared for: Development / Product Security Team

Role: Independent Security Researcher (Intern)

Assessment Type: Black-Box Penetration Test

Target: OWASP Juice Shop (local instance) — <http://127.0.0.1:3000>

Report Date: 12 August 2026

Classification: Confidential

1. Executive Summary

Between 11 and 12 August 2026, a black-box penetration test was performed against a locally hosted instance of OWASP Juice Shop, simulating an external attacker with no prior knowledge of the application's source code, database structure, or user accounts. Testing was conducted manually using Burp Suite Professional, the Firefox/Chromium browser and developer tools, following a standard web-application penetration testing methodology (reconnaissance, mapping, authenticated and unauthenticated testing, manual exploitation, and reporting).

In accordance with the rules of engagement, the built-in Juice Shop scoreboard was not consulted at any point; all findings below were independently discovered and manually verified with a working proof-of-concept. A total of six vulnerabilities were identified spanning multiple OWASP Top 10 categories, ranging from Critical (full authentication bypass) to Low severity (information disclosure). The most severe finding allows any unauthenticated attacker to log in as the site administrator via SQL Injection. Detailed reproduction steps, evidence, impact analysis, and remediation guidance for every finding are provided in Section 4.

1.1 Severity Breakdown

Critical	High	Medium	Low
1	2	2	1

2. Scope & Rules of Engagement

2.1 In Scope

- All endpoints, functionality, and user roles on the local OWASP Juice Shop instance (<http://127.0.0.1:3000>).

2.2 Out of Scope

- The Juice Shop built-in scoreboard page — not used to guide or discover findings.

2.3 Tools Used

- Burp Suite Professional (Proxy, Repeater, Intruder)
- Browser Developer Tools (Chromium)
- Manual testing and hand-crafted payloads

2.4 Methodology

Testing followed a standard black-box methodology:

- (1) unauthenticated reconnaissance and normal-user walkthrough (registration, browsing, cart, checkout);
- (2) mapping of REST API endpoints via the intercepting proxy;
- (3) authenticated and unauthenticated manual testing for each OWASP Top 10 category;
- (4) manual exploitation and proof-of-concept capture for every confirmed issue;
- (5) CVSS v3.1 scoring and reporting.

3. Summary of Findings

#	Finding	Severity	CVSS v3	Affected Endpoint
1	SQL Injection Leading to Authentication Bypass (Admin Account Takeover)	Critical	9.8	POST /rest/user/login
2	Broken Access Control (IDOR) — Unauthorized Access to Other Users' Shopping Baskets	High	8.1	GET /rest/basket/{id}
3	Reflected Cross-Site Scripting (XSS) in Product Search	Medium	6.1	GET /#/search?q=
4	Sensitive/Confidential Information Disclosure via Exposed FTP Directory	High	7.5	GET /ftp/ and /ftp/acquisitions.md
5	Verbose Error Messages / Stack Trace Disclosure	Medium	5.3	GET /rest/{invalid-path}
6	Exposed Application Metrics Endpoint (Information Disclosure)	Low	5.3	GET /metrics

4. Detailed Findings

4.1 SQL Injection Leading to Authentication Bypass (Admin Account Takeover)

Severity	Critical
CVSS v3.1 Score	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Affected Endpoint	POST /rest/user/login

Description

The login endpoint concatenates the user-supplied 'email' field directly into a backend SQL query used to authenticate users, without parameterization or input sanitization. By submitting a classic SQL Injection payload in the email field, an attacker can alter the logic of the WHERE clause so that it always evaluates to true, causing the application to authenticate as the first matching user in the database — which, in Juice Shop, is the administrator account. No valid credentials are required.

Steps to Reproduce

1. Navigate to the login page and intercept the login request in Burp Suite Proxy/Repeater.
2. Replace the JSON body with: {"email":""," OR 1=1--","password":"test"}
3. Send the request to POST /rest/user/login.
4. Observe the HTTP 200 response: the server returns a valid JWT authentication token and the 'email' field is populated with 'admin@juice-sh.op', confirming the attacker is now authenticated as the administrator without knowing the admin password.

Proof of Concept

```
POST /rest/user/login HTTP/1.1
Host: 127.0.0.1:3000
Content-Type: application/json

{"email":"' OR 1=1--","password":"test"}

-- Server Response (200 OK) --
{"authentication":{"token":"eyJ...<JWT>...", "bid":1,
  "umail":"admin@juice-sh.op"}}
```

Impact

Complete authentication bypass allows any unauthenticated attacker to obtain a valid session as the site administrator. This grants access to all administrative functionality, other users' personal data, order history, and potentially full control over store content and configuration. This is a Critical, full-compromise-class vulnerability under OWASP Top 10 2025 category A03 (Injection).

Remediation

- Use parameterized queries / prepared statements (e.g. Sequelize bind parameters) for all database access — never concatenate user input into SQL strings.
- Apply strict server-side input validation on the email field (valid email format only).
- Enforce account lockout / rate limiting on the login endpoint to slow brute-force and injection probing.
- Run static analysis (SAST) and periodic dependency/DB-query audits to catch string-concatenated queries before release.

4.2 Broken Access Control (IDOR) — Unauthorized Access to Other Users' Shopping Baskets

Severity	High
CVSS v3.1 Score	8.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N
Affected Endpoint	GET /rest/basket/{id}

Description

The /rest/basket/{id} endpoint returns basket contents based solely on the numeric ID supplied in the URL path. The server does not verify that the basket ID belongs to the currently authenticated user (whose identity is carried in the JWT bearer token). By simply incrementing/changing the basket ID, an authenticated low-privileged user can retrieve any other customer's basket contents — a textbook Insecure Direct Object Reference (IDOR).

Steps to Reproduce

1. Log in as a low-privileged test user and add an item to the basket to obtain a valid Bearer token.
2. Send GET /rest/basket/6 with the current user's Authorization: Bearer token via Burp Repeater. Note the response contains "userId":25.
3. Without changing the Authorization token, change the URL to GET /rest/basket/7.
4. Observe the server returns HTTP 200 with basket data belonging to a completely different account, "userId":27, including that user's product selections.
5. Repeat with sequential IDs to confirm the pattern is enumerable across all baskets in the system.

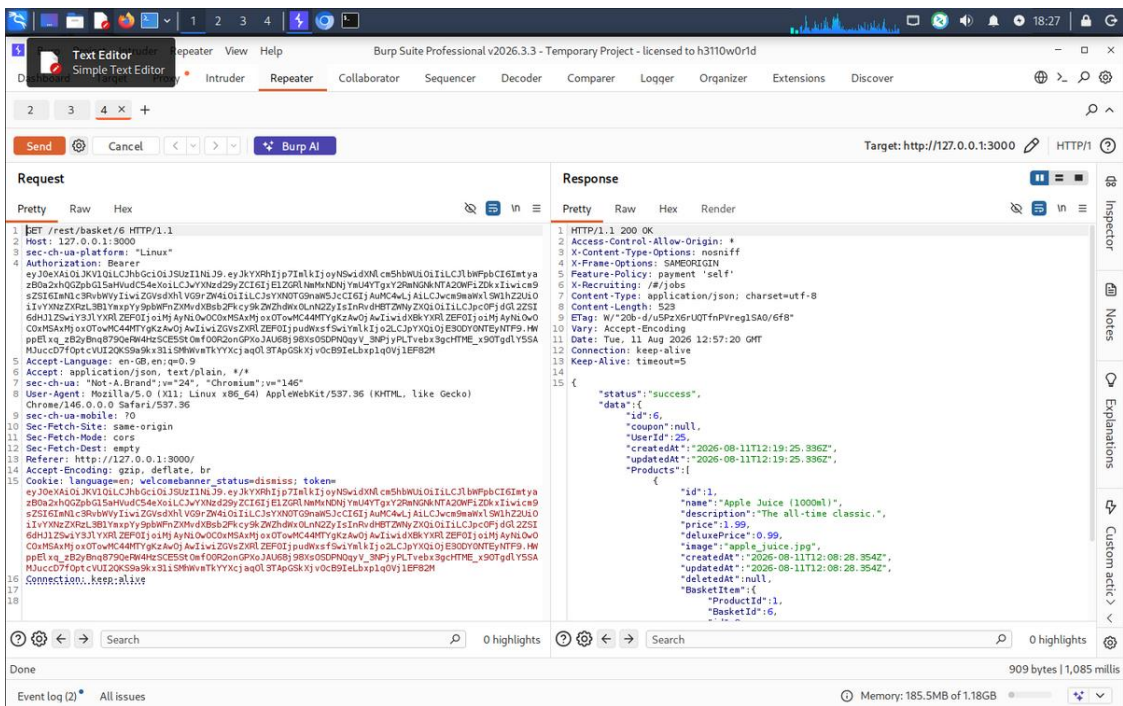


Figure 2.1 — Basket 6 belongs to userId 25 (the requesting user's own basket)

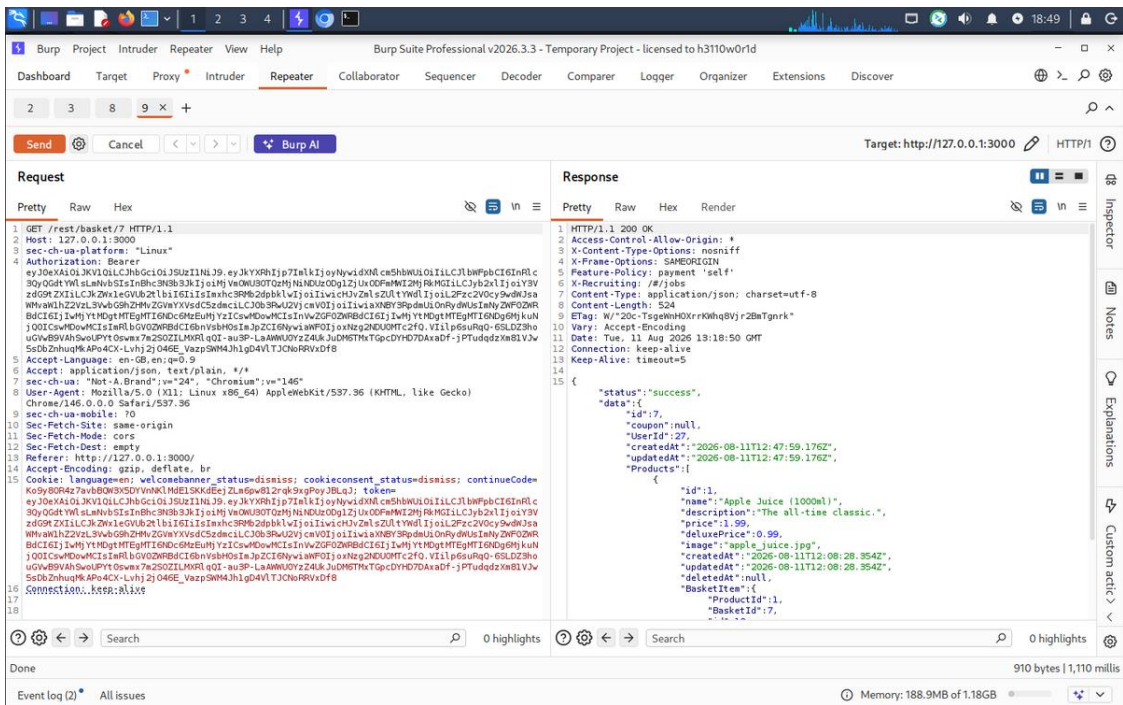


Figure 2.2 — Same Bearer token retrieves basket 7, belonging to a different userId 27

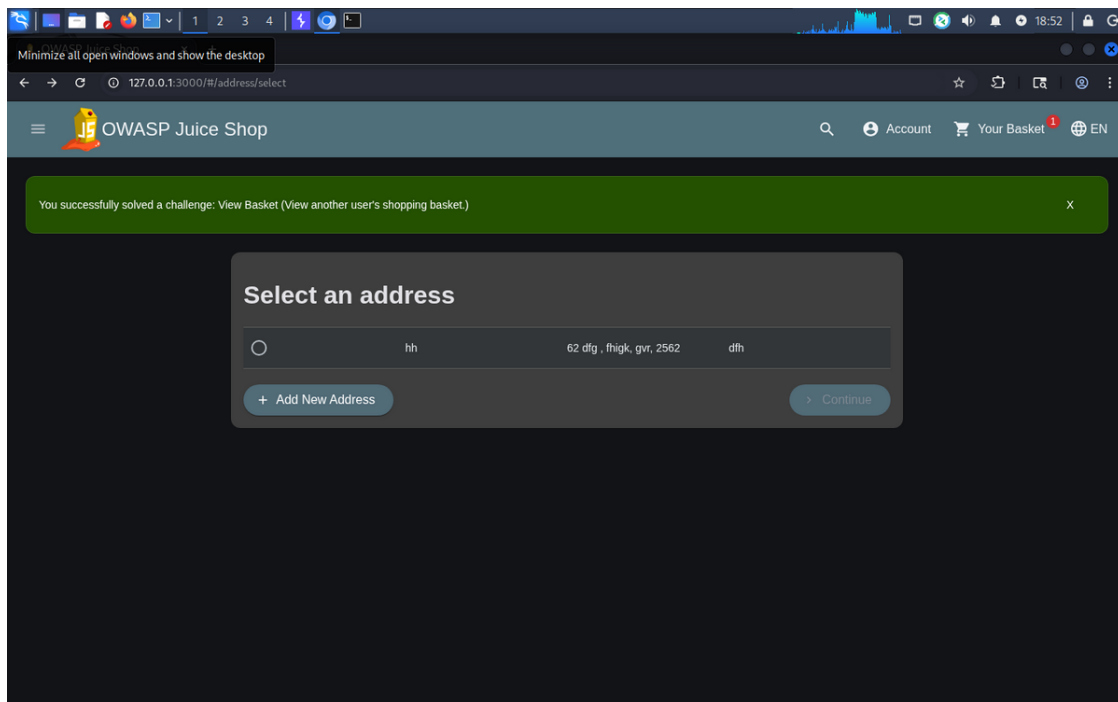


Figure 2.3 — Application confirms 'View Basket' challenge solved

Proof of Concept

```
GET /rest/basket/7 HTTP/1.1
Host: 127.0.0.1:3000
Authorization: Bearer <token belonging to userId 25>

-- Server Response (200 OK) --
{"status":"success","data":{"id":7,"userId":27,
  "Products":[{"id":1,"name":"Apple Juice (1000ml)", ...}]}}
```

Impact

Any authenticated user can enumerate and view the private basket contents of every other user on the platform simply by changing an integer in the URL. This exposes personal purchasing data and, depending on downstream checkout logic, could be chained with basket manipulation to view or interfere with other users' orders. Maps to OWASP Top 10 2025 A01 (Broken Access Control).

Remediation

- Enforce server-side ownership checks: verify the basket's userId matches the userId embedded in the caller's JWT before returning data.
- Replace sequential, guessable numeric IDs with non-enumerable identifiers (UUIDs) for basket/order resources.
- Add automated authorization tests (BOLA/IDOR regression tests) to the CI pipeline for every object-level endpoint.

4.3 Reflected Cross-Site Scripting (XSS) in Product Search

Severity	Medium
CVSS v3.1 Score	6.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
Affected Endpoint	GET /#/search?q=

Description

The product search feature reflects the 'q' query parameter back into the DOM without adequate output encoding/sanitization. Submitting an HTML/JavaScript payload as a search term causes the browser to render attacker-controlled markup, resulting in arbitrary JavaScript execution in the context of the victim's session (DOM-based / reflected XSS).

Steps to Reproduce

1. Open the search bar and enter the payload: `<iframe src="javascript:alert('xss')">`
2. Submit the search (or navigate directly to the crafted URL: `/#/search?q=<iframe%20src%3D"javascript:alert('xss')">`).
3. Observe that the browser renders the injected iframe and executes the embedded JavaScript, popping a native `alert()` dialog — proving unsanitized HTML injection into the DOM.

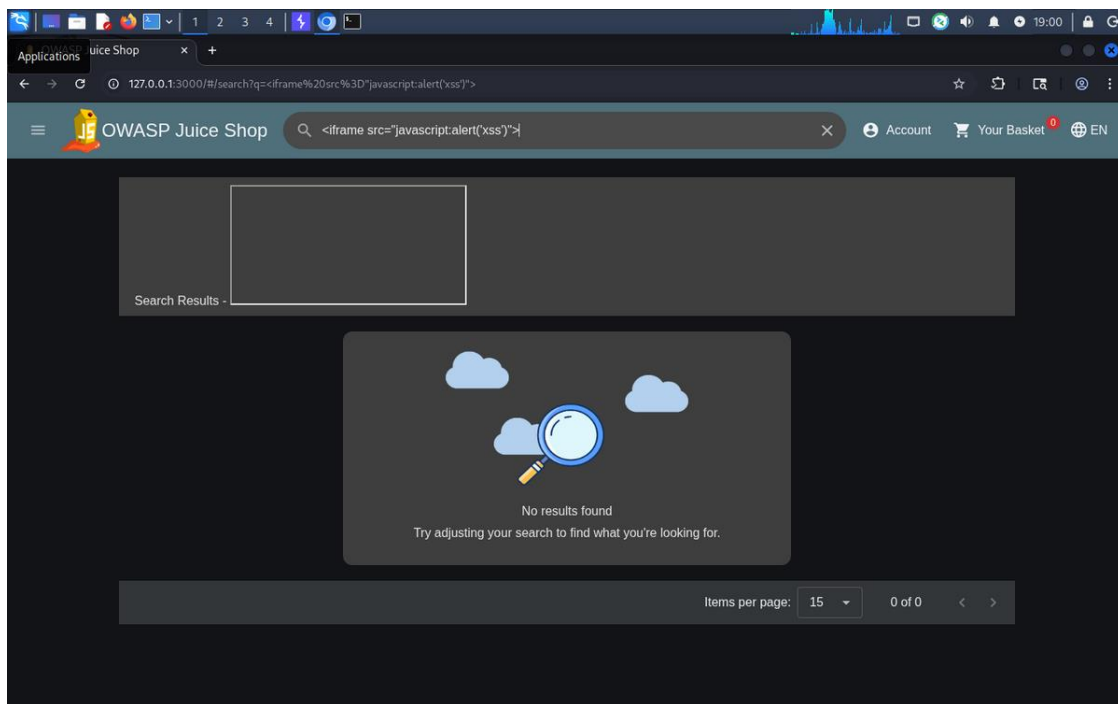


Figure 3.1 — XSS payload entered into the search field

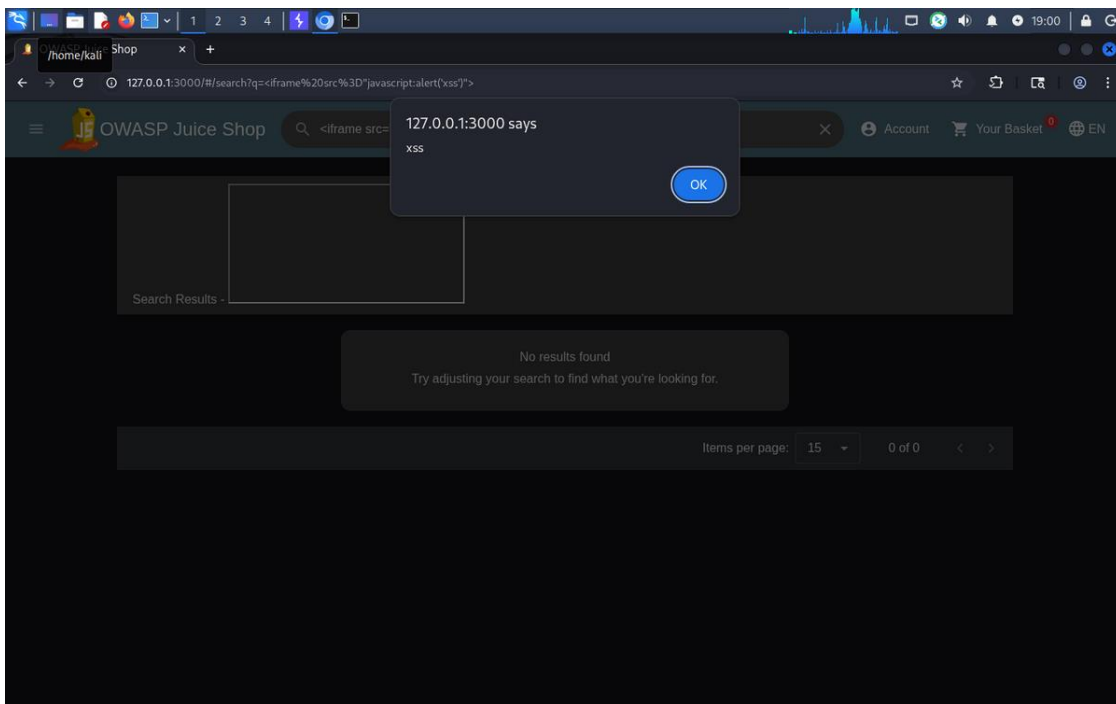


Figure 3.2 — JavaScript alert() executes, confirming reflected XSS

Proof of Concept

URL:

`http://127.0.0.1:3000/#/search?q=<iframe%20src="javascript:alert('xss')">`

Payload:

`<iframe src="javascript:alert('xss')">`

Impact

An attacker can craft a malicious search link and distribute it (phishing email, forum post, chat) to victims. When a logged-in victim opens the link, arbitrary JavaScript executes in their authenticated session — enabling session/token theft, forced actions on the user's behalf, credential harvesting via fake login overlays, or redirection to malicious sites. Maps to OWASP Top 10 2025 A03 (Injection) / XSS.

Remediation

- Encode all user-supplied data before rendering it into the DOM (use framework-safe binding, e.g. Angular's built-in sanitizer, and avoid `innerHTML`/`bypassSecurityTrust` APIs for untrusted input).
- Implement a strict Content-Security-Policy (CSP) header to prevent inline script/iframe execution.
- Apply server-side input validation/allow-listing on the search parameter in addition to client-side encoding.

4.4 Sensitive/Confidential Information Disclosure via Exposed FTP Directory

Severity	High
CVSS v3.1 Score	7.5
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Affected Endpoint	GET /ftp/ and /ftp/acquisitions.md

Description

The web application exposes an unauthenticated, browsable directory at /ftp containing internal company files (legal documents, backup/config files, an encrypted announcement, and — critically — a KeePass database file). One of the listed files, acquisitions.md, is a plain-text markdown document explicitly marked confidential and containing non-public information about planned company acquisitions. No authentication or access control restricts access to this directory or its contents.

Steps to Reproduce

1. Browse directly to <http://127.0.0.1:3000/ftp> with no authentication.
2. Observe a directory listing of internal files, including acquisitions.md, legal.md, incident-support.kdbx (a KeePass password database), and backup files (package.json.bak, package-lock.json.bak).
3. Open <http://127.0.0.1:3000/ftp/acquisitions.md> directly.
4. Observe the file is served in full and is explicitly labelled '# Planned Acquisitions' / 'This document is confidential! Do not distribute!' — confirming unauthorized disclosure of sensitive business information.

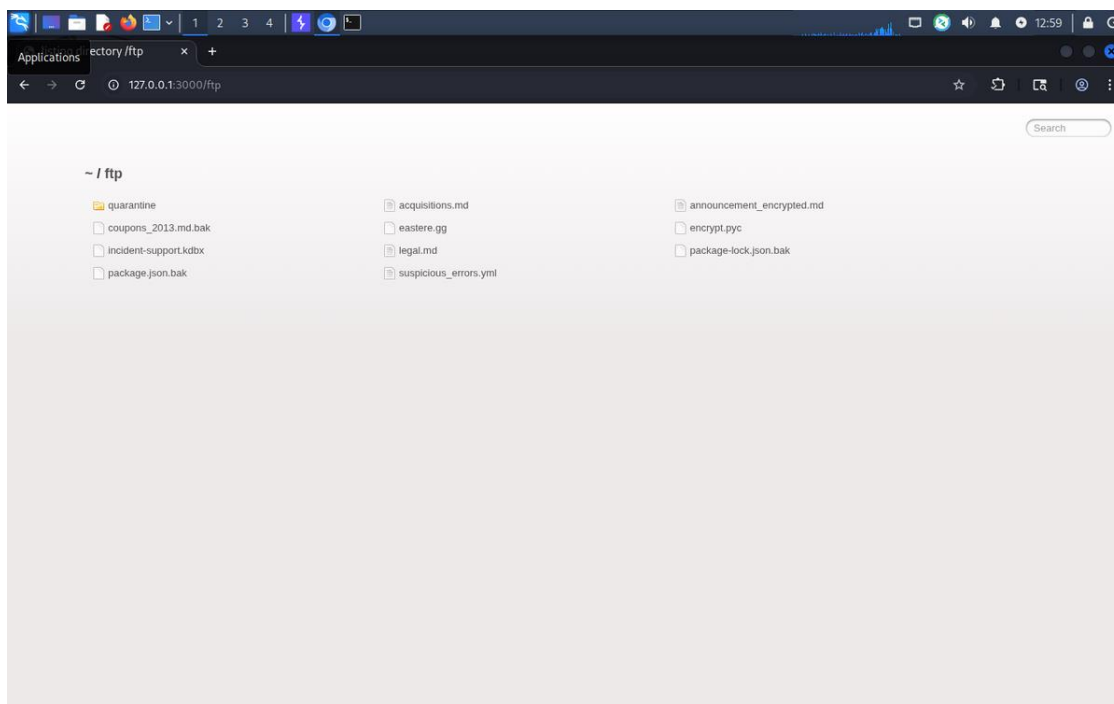


Figure 4.1 — Unauthenticated directory listing at /ftp exposing internal files

Proof of Concept

```
GET /ftp/acquisitions.md HTTP/1.1
Host: 127.0.0.1:3000

-- Server Response (200 OK) --
# Planned Acquisitions
> This document is confidential! Do not distribute!
Our company plans to acquire several competitors within the next year...
```

Impact

Unauthenticated attackers can retrieve confidential business documents (M&A plans, legal files) and sensitive artifacts such as a KeePass credential database, which could be brute-forced offline to recover stored passwords. Disclosure of acquisition plans could cause material financial/reputational harm and constitutes a serious breach of confidentiality. Maps to OWASP Top 10 2025 A05 (Security Misconfiguration) / Sensitive Data Exposure.

Remediation

- Remove the /ftp directory (and any legacy/staging files) from the production web root entirely; it should never be reachable via the public web server.
- If legacy file transfer is genuinely required, move it off the web server and behind authentication + network-level restrictions.
- Audit the web root for other leftover backup (.bak), credential (.kdbx), and internal documentation files and purge them.
- Add directory-listing disabling and deny-by-default rules at the web server/reverse-proxy layer.

4.5 Verbose Error Messages / Stack Trace Disclosure

Severity	Medium
CVSS v3.1 Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Affected Endpoint	GET /rest/{invalid-path}

Description

Requesting an undefined REST route (e.g. a random, non-existent path under /rest/) causes the application to return an HTTP 500 error page containing a full server-side stack trace. The disclosed information includes the exact Express.js framework version, internal file system paths (/juice-shop/build/routes/..., /juice-shop/node_modules/...), and the internal routing/middleware call chain.

Steps to Reproduce

1. Send a GET request to a non-existent REST path, e.g. GET /rest/qwertz.
2. Observe the HTTP 500 response rendered directly in the browser and confirmed in Burp Repeater.
3. Note the page discloses 'OWASP Juice Shop (Express ^4.22.1)' and a full Node.js/Express stack trace revealing internal file paths and library versions.

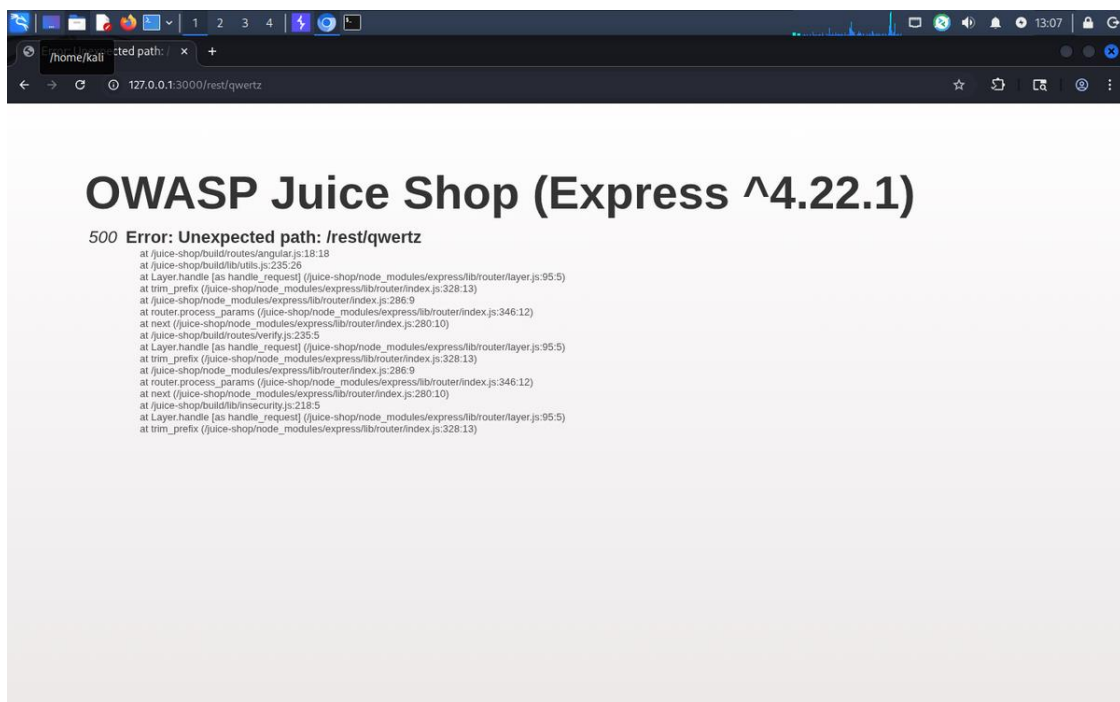


Figure 5.1 — Full stack trace and framework version rendered to an unauthenticated browser

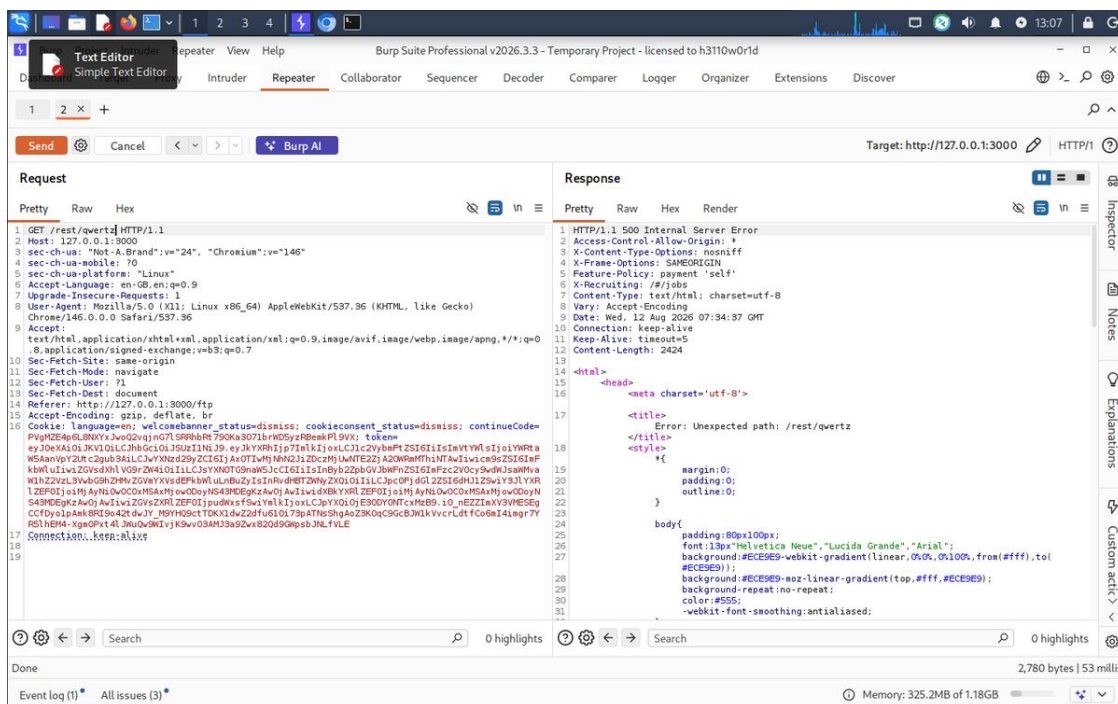


Figure 5.2 — Burp Repeater capture of the HTTP 500 response with internal file paths

Proof of Concept

```
GET /rest/qwertz HTTP/1.1
Host: 127.0.0.1:3000

-- Server Response (500 Internal Server Error) --
OWASP Juice Shop (Express ^4.22.1)
500 Error: Unexpected path: /rest/qwertz
  at /juice-shop/build/routes/angular.js:18:18
  at /juice-shop/build/lib/utlis.js:235:26
  at Layer.handle [as handle_request] (/juice-shop/node_modules/express/...)
```

Impact

While not directly exploitable on its own, this disclosure gives an attacker precise framework/library version information and internal application structure, which significantly accelerates reconnaissance for targeted exploit selection (e.g. checking for known CVEs against the disclosed Express version) and reduces the effort needed to map the application's internals. Maps to OWASP Top 10 2025 A05 (Security Misconfiguration).

Remediation

- Disable detailed error output in production (set `NODE_ENV=production` and use a generic error-handling middleware that returns a sanitized message).
- Log full stack traces server-side only (to a monitoring/SIEM system), never return them to the client.
- Return a consistent, generic error response (e.g. HTTP 404/500 with a minimal JSON body) for unmatched routes and unhandled exceptions.

4.6 Exposed Application Metrics Endpoint (Information Disclosure)

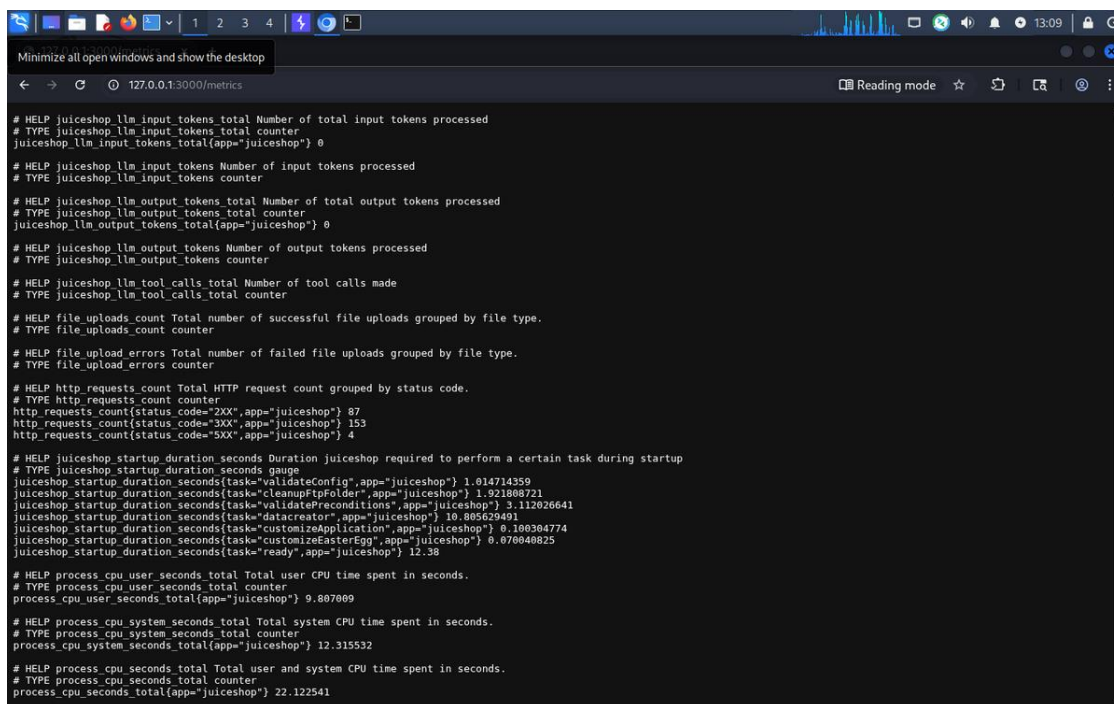
Severity	Low
CVSS v3.1 Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Affected Endpoint	GET /metrics

Description

The application exposes a Prometheus-style /metrics endpoint publicly, with no authentication required. This endpoint discloses internal operational telemetry, including HTTP request counts by status code (2XX/3XX/5XX), process CPU usage, startup task durations, and internal task names (e.g. cleanupFtpFolder, datacreator, customizeEasterEgg) that reveal implementation details of the application's startup and maintenance routines.

Steps to Reproduce

1. Send an unauthenticated GET request to /metrics.
2. Observe HTTP 200 response containing detailed Prometheus metrics for the running application instance.
3. Note that internal task names, request volume by status code, and CPU/process statistics are all readable by any anonymous visitor.



```
# HELP juiceshop_llm_input_tokens_total Number of total input tokens processed
# TYPE juiceshop_llm_input_tokens_total counter
juiceshop_llm_input_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_input_tokens Number of input tokens processed
# TYPE juiceshop_llm_input_tokens counter

# HELP juiceshop_llm_output_tokens_total Number of total output tokens processed
# TYPE juiceshop_llm_output_tokens_total counter
juiceshop_llm_output_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_output_tokens Number of output tokens processed
# TYPE juiceshop_llm_output_tokens counter

# HELP juiceshop_llm_tool_calls_total Number of tool calls made
# TYPE juiceshop_llm_tool_calls_total counter

# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter

# HELP http_requests_count Total HTTP request count grouped by status code.
# TYPE http_requests_count counter
http_requests_count{status_code="2XX",app="juiceshop"} 87
http_requests_count{status_code="3XX",app="juiceshop"} 153
http_requests_count{status_code="5XX",app="juiceshop"} 4

# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds{task="validateConfig",app="juiceshop"} 1.014714359
juiceshop_startup_duration_seconds{task="cleanupFtpFolder",app="juiceshop"} 1.921808721
juiceshop_startup_duration_seconds{task="validatePreconditions",app="juiceshop"} 3.112026641
juiceshop_startup_duration_seconds{task="datacreator",app="juiceshop"} 10.005629491
juiceshop_startup_duration_seconds{task="customizeApplication",app="juiceshop"} 0.100304774
juiceshop_startup_duration_seconds{task="customizeEasterEgg",app="juiceshop"} 0.070040825
juiceshop_startup_duration_seconds{task="ready",app="juiceshop"} 12.38

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total{app="juiceshop"} 9.807009

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total{app="juiceshop"} 12.315532

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total{app="juiceshop"} 22.122541
```

Figure 6.1 — /metrics reachable directly in the browser without authentication

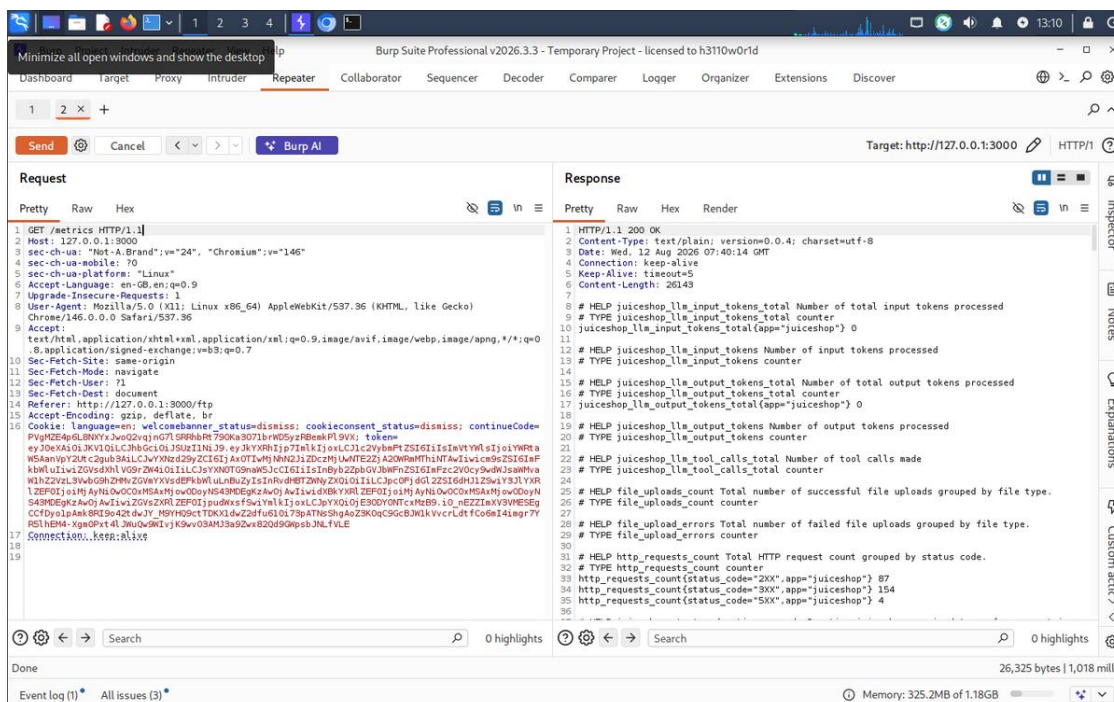


Figure 6.2 — Burp Repeater capture confirming HTTP 200 and full metrics payload

Proof of Concept

```
GET /metrics HTTP/1.1
Host: 127.0.0.1:3000

-- Server Response (200 OK) --
http_requests_count{status_code="2XX",app="juiceshop"} 87
http_requests_count{status_code="5XX",app="juiceshop"} 4
juiceshop_startup_duration_seconds{task="cleanupFtpFolder"} 1.92
process_cpu_seconds_total{app="juiceshop"} 22.12
```

Impact

Publicly exposed metrics leak operational intelligence that aids an attacker in fingerprinting the application, gauging traffic/error patterns to time attacks (e.g. spikes in 5XX errors indicating instability), and learning about internal maintenance jobs. While the individual data points are low-severity, in aggregate they meaningfully assist reconnaissance and should not be publicly reachable. Maps to OWASP Top 10 2025 A05 (Security Misconfiguration).

Remediation

- Restrict /metrics to internal monitoring infrastructure only (bind to localhost/internal network, or require authentication/mTLS for scraping).
- Place the metrics endpoint behind the reverse proxy with an IP allow-list limited to the monitoring system.
- Review all operational/debug endpoints for similar unauthenticated exposure as part of a security misconfiguration audit.

5. Conclusion & Overall Recommendations

This engagement identified six confirmed vulnerabilities across authentication, access control, injection, and information-disclosure categories. The single highest-priority item is the SQL Injection authentication bypass (Finding 4.1), which should be remediated immediately as it grants unauthenticated attackers full administrative access. The Broken Access Control / IDOR issue (4.2) and Confidential Information Disclosure via the exposed /ftp path (4.4) should be treated as the next priorities, followed by the Medium and Low severity issues.

Beyond the individual fixes listed per finding, we recommend the development team: (1) adopt parameterized queries/ORMs consistently across all data access code; (2) implement object-level authorization checks on every endpoint that accepts a resource ID; (3) remove all legacy/staging files and directories from the production web root; (4) disable verbose error output and restrict internal telemetry endpoints in production; and (5) incorporate automated security testing (SAST/DAST) and IDOR/BOLA regression tests into the CI/CD pipeline going forward.